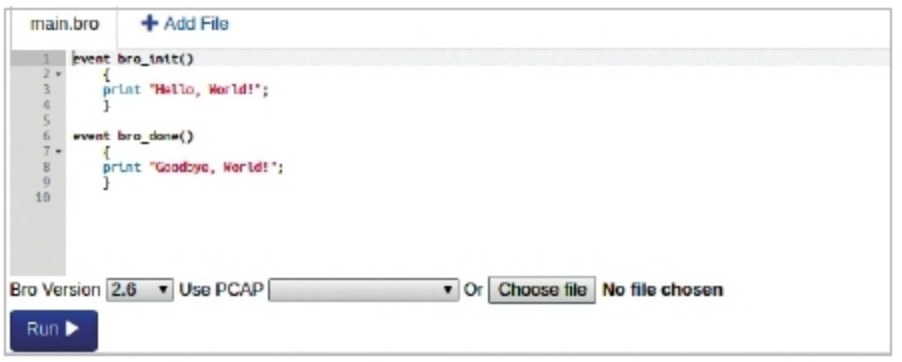

Figure 4: HelloWorld

To start learning Zeek, the easiest way is to use the interactive tutorial available at *http://try.zeek.org*. A detailed tutorial with an extensive list of examples is available at *https://www.zeek.org/documentation/tutorials/index.html*.

HelloWorld

try.zeek.org makes it very easy to try out Zeek's functionalities. A screenshot of *try.bro.org* is shown in Figure 4.

You can simply enter the Zeek script in the code window and click on *Run*. You can see the output appearing in the output box.

Note that the documentation and code uses the word Bro instead of Zeek. Throughout the documentation and official website, the words Bro and Zeek are used in tandem.

```
event bro_init()
     {
     print "Hello, World!";
     }

event bro_done()
     {
     print "Goodbye, World!";
     }
```

As stated earlier, Zeek is event-driven. There are two Zeek events that are raised always. They are *bro_init()* and *bro_done()*. *bro_init()* is executed when Zeek is started and *bro_done()* is called when it terminates.

A code example with the function calling in Zeek script is given below:

```
# Function implementation.
function emphasize(s: string, p: string &default = "*"):
string
     {
     return p + s + p;
     }
event bro_init()
     {
     # Function calls.
     print emphasize("yes");
     print emphasize("no", "_");
     }
```

A code snippet to use looping with Zeek is also shown below:

```
event bro_init()
     {
     for ( character in "OSFY" )
       {
       print character;
       }
     }

The output of the above code is:

O
S
F
Y
```

A simple logging example with Bro script is as shown below. It makes two logs — one for Mod 5 and another for non-Mod 5.

```
@load factorial

event bro_init()
    {
    Log::create_stream(Factor::LOG, [$columns=Factor::Info,
$path="factor"]);

    local filter: Log::Filter = [$name="split-mod5s", $path_
func=Factor::mod5];
    Log::add_filter(Factor::LOG, filter);
    Log::remove_filter(Factor::LOG, "default");
    }

event bro_done()
    {
    local numbers: vector of count = vector(1, 2, 3, 4, 5, 6,
7, 8, 9, 10);
    for ( n in numbers)
        Log::write( Factor::LOG, [$num=numbers[n],
                                  $factorial_num=Factor::fact
orial(numbers[n])]);
    }
```

A code snippet to raise a notice for specific events is as shown below:

```
@load factorial

event bro_init()
    {
    Log::create_stream(Factor::LOG, [$columns=Factor::Info,
$path="factor"]);
```